Table of contents

Model Checking

Introduction to Model Checking

Model checking is a formal verification technique for systems, proposed in 1981 by Clarke, Emerson, and Sifakis. It allows automatically verifying whether a system (modeled by a finite-state automaton) satisfies a property specified in temporal logic.

The Model Checking Problem

Given a finite automaton A (possibly with labels) and a temporal logic formula F, the problem is to:

- Determine if $A \models F$, i.e., if all traces of A are included in the traces of F: $Traces(A) \subseteq Traces(F)$.

- If not, provide a counterexample: an execution of A that does not satisfy F.

The difficulty of the problem (decidability, complexity) depends on the expressiveness of A and F. In this course, we focus on:

- A: a finite automaton with only actions (simple labels) on transitions, and nothing on states.
- F: a linear temporal logic (LTL) formula.

Infinite Words and -Regular Languages

-Words

Reactive systems do not stop, so their executions are infinite. Similarly, the properties to verify concern infinite behaviors. An infinite trace (or word) of actions is called an **-word**.

Example: The infinite word consisting of the infinite repetition of "ab" is written $(ab)^\omega$.

Compare with $(ab)^*$ which represents finite words with an arbitrary (finite) number of repetitions of "ab". The symbol ω indicates an infinite repetition.

-Regular Expressions

By extending regular expressions with the operator ω, we define **-regular expressions**.

Examples: Over the alphabet $\{a, b\}$:

- $(a + b)^*.a^\omega$ represents infinite words with a finite number of b.
- $(a^*b)^\omega$ represents infinite words with an infinite number of b.

Caution: $a^\omega b^\omega$ does not make sense (we cannot finish reading the infinite a to move to b) and is therefore forbidden.

-Regular Languages

Just as regular languages can be represented by regular expressions or finite automata, **-regular languages** are exactly those that can be expressed by:

- -regular expressions (with operators $+, \cdot, *, \omega$),
- Büchi automata.

Definition: A language L is -regular if it can be written in one of the following forms (and their combinations):

1. $L = A^\omega$ where A is a regular language and $\epsilon \notin A$,
2. $L = A \cdot B$ where A is regular and B is -regular,
3. $L = A \cup B$ where A and B are -regular.

Closure properties: If we have automata (finite or Büchi) for A and B, we can construct a Büchi automaton for A^ω, $A \cdot B$, $A + B$, $A \cap B$. However, complementation $\overline{A}$ is more complex (exponential size).

Important remark: Unlike finite automata, a nondeterministic Büchi automaton cannot always be determinized. For example, $(a+b)^* a^\omega$ cannot be recognized by a deterministic Büchi automaton.

Emptiness Problem for -Regular Languages

The problem of deciding whether an -regular language L is empty is **decidable**. Given a Büchi automaton A such that $L(A) = L$, we have:

$L \neq \emptyset$ if and only if A contains an -word. Since the number of final states is finite, every accepted -word is ultimately periodic.

Thus, $L \neq \emptyset$ iff A contains a strongly connected component (SCC) reachable from the initial state and containing a final state.

Büchi Automata

Formal Definition

A Büchi automaton is a tuple $\mathcal{A} = (S, S_0, L, T, F)$ where:

- S: finite set of states,
- $S_0 \subseteq S$: initial states,

- L: alphabet (labels),
- $T \subseteq S \times L \times S$: transition relation,
- $F \subseteq S$: final states (or acceptance states).

Acceptance Semantics

An -word $w = a_0 a_1 a_2 \ldots$ is accepted by $\mathcal{A}$ if there exists an infinite execution $s_0 s_1 s_2 \ldots$ such that:

- $s_0 \in S_0$,
- for all $i \geq 0$, $(s_i, a_i, s_{i+1}) \in T$,
- and **a final state $f \in F$ appears infinitely often** in the execution.

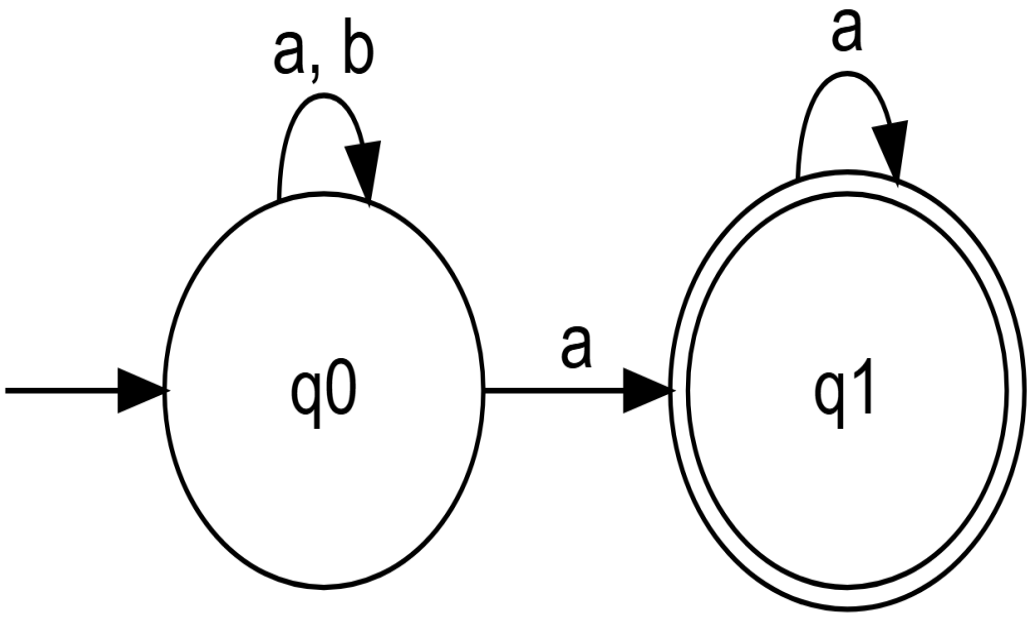
The language recognized by $\mathcal{A}$, denoted $L(\mathcal{A})$, is the set of accepted -words.

Remark: This acceptance condition (visiting a final state infinitely often) is characteristic of Büchi automata.

Here is an example of a Büchi automaton illustrating the -regular language $(a+b)^* a^\omega$ (infinite words containing a finite number of b), integrated directly into your course.

Example

The Büchi automaton below recognizes the -regular language $L = (a+b)^* a^\omega$, i.e., the set of infinite words over $\{a, b\}$ that, after some finite prefix, contain only the symbol a.



Operation of the automaton:

- From the initial state q_0, the automaton can read any finite sequence (possibly empty) of a and b while staying in q_0 (transition on a, b).
- At any time, upon reading an a, it can choose to go to the final state q_1.
- Once in q_1, it can only read a (loop on a) to stay there.
- For an infinite word to be **accepted**, there must exist an execution that visits q_1 infinitely often. The only way to do this is to go to q_1 at some point and then read only a indefinitely. Therefore, any accepted word must end with an infinity of a, which exactly corresponds to the definition of the language $(a + b)^* a^\omega$.

This example also illustrates the **nondeterminism** of some Büchi automata: upon reading an a in q_0, the automaton can choose between staying in q_0 or going to q_1. This nondeterminism is essential to recognize this language with a Büchi automaton.

Linear Temporal Logic (LTL)

Introduced by Pnueli in 1977, LTL allows expressing properties about the infinite behaviors of systems.

Syntax

The syntax of LTL is defined by the following grammar:

$$\phi ::= a \mid \neg\phi \mid \phi_1 \vee \phi_2 \mid \bigcirc\phi \mid \phi_1 \ \mathcal{U} \ \phi_2$$

where a is a propositional variable (an action or atomic property).

Semantics

An LTL formula ϕ interprets an -word $\sigma = (\sigma_i)_{i\geq 0}$ (where each σ_i is a set of propositions true at instant i). The satisfaction relation $\sigma \vDash \phi$ is defined inductively:

- $\sigma \vDash a$ iff $a \in \sigma_0$ (the proposition a is true at the first instant).
- $\sigma \vDash \neg\phi$ iff $\sigma \nvDash \phi$.
- $\sigma \vDash \phi_1 \vee \phi_2$ iff $\sigma \vDash \phi_1$ or $\sigma \vDash \phi_2$.
- $\sigma \vDash \bigcirc\phi$ iff $\sigma' = (\sigma_i)_{i\geq 1} \vDash \phi$ (the formula ϕ is true at the next instant).
- $\sigma \vDash \phi_1 \ \mathcal{U} \ \phi_2$ iff there exists $j \geq 0$ such that:
  - $\sigma^{(j)} = (\sigma_i)_{i\geq j} \vDash \phi_2$,
  - and for all k with $0 \leq k < j$, we have $\sigma^{(k)} = (\sigma_i)_{i\geq k} \vDash \phi_1$.

Intuitively, $\phi_1 \ \mathcal{U} \ \phi_2$ means that ϕ_1 remains true until ϕ_2 becomes true (and ϕ_2 eventually becomes true).

Derived Operators

From the basic operators, we define:

- **Eventually:** $\Diamond\phi \equiv$ True $\mathcal{U} \ \phi$. The formula ϕ becomes true after a finite time.
- **Always:** $\Box\phi \equiv \neg\Diamond\neg\phi$. The formula ϕ is true at every instant.

Some useful equivalences:

- $\Box(a \vee b) \neq \Box a \vee \Box b$
- $\Diamond(a \vee b) \equiv \Diamond a \vee \Diamond b$
- $\Box(a \wedge b) \equiv \Box a \wedge \Box b$
- $\Diamond(a \wedge b) \neq \Diamond a \wedge \Diamond b$ (since a and b can occur at different instants).

LTL and Büchi Automata

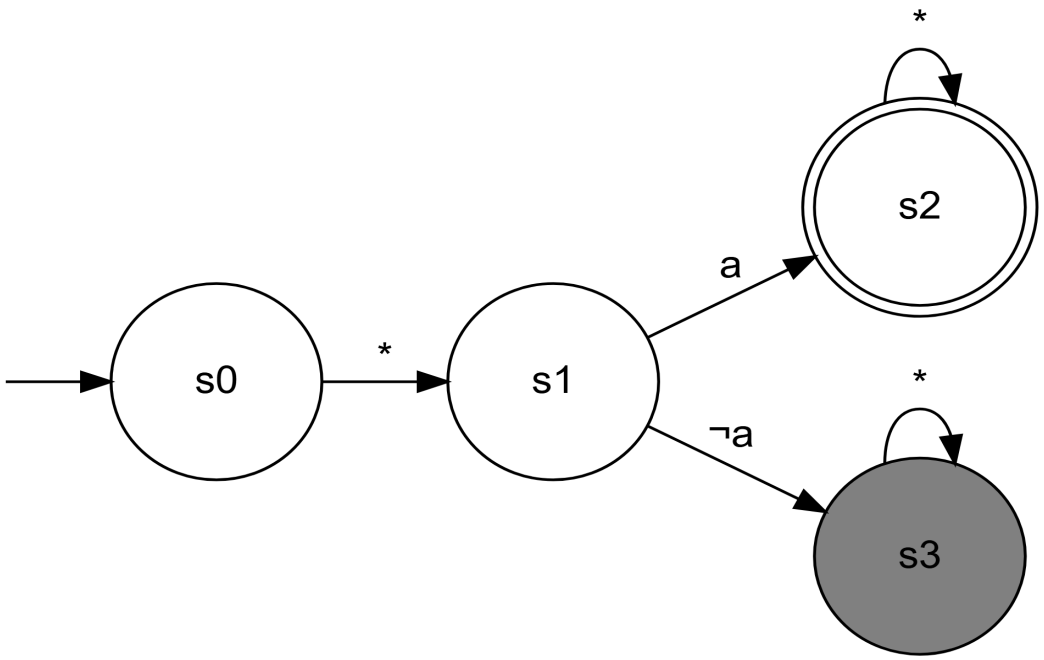
A fundamental result: every LTL formula can be translated into a Büchi automaton that accepts exactly the -words satisfying the formula. Algorithms exist for this construction.

Büchi Automata for Basic LTL Operators

To better understand the semantics of LTL operators, here are Büchi automata that recognize exactly the -words satisfying each formula.

1. Next Operator: $\bigcirc a$

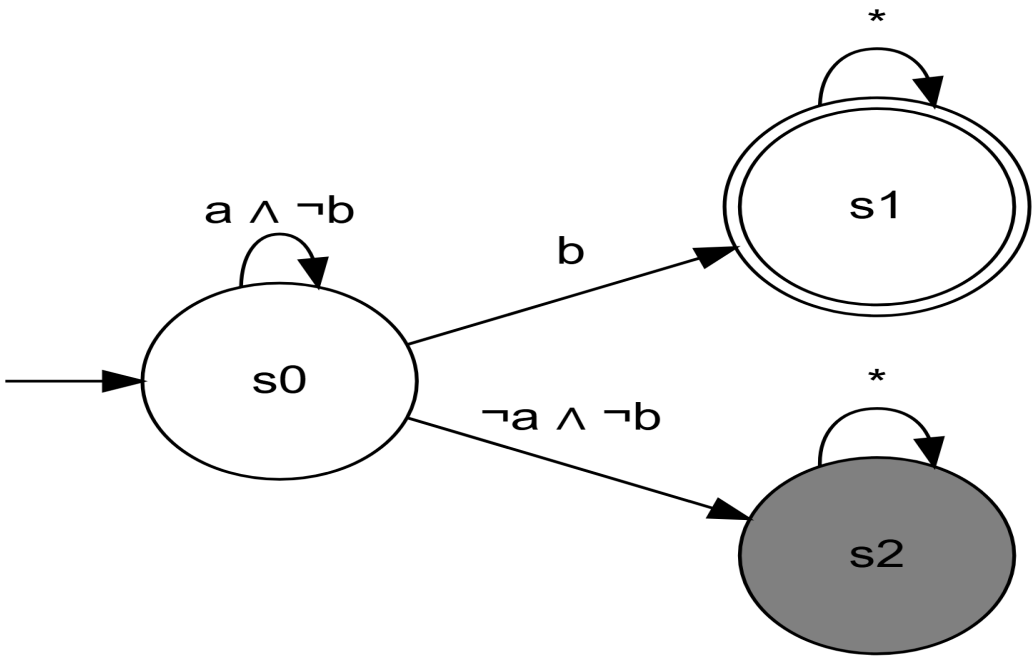
The formula $\bigcirc a$ requires that a be true at the instant following the current instant.



Operation: The automaton immediately moves from the initial state s_0 to s_1 (representing the next instant). If a is true in this next state (s_1), it goes to the final state s_2 and the word is accepted. If a is false, it enters the sink state s_3 and is not accepted.

2. Until Operator: $a \ \mathcal{U} \ b$

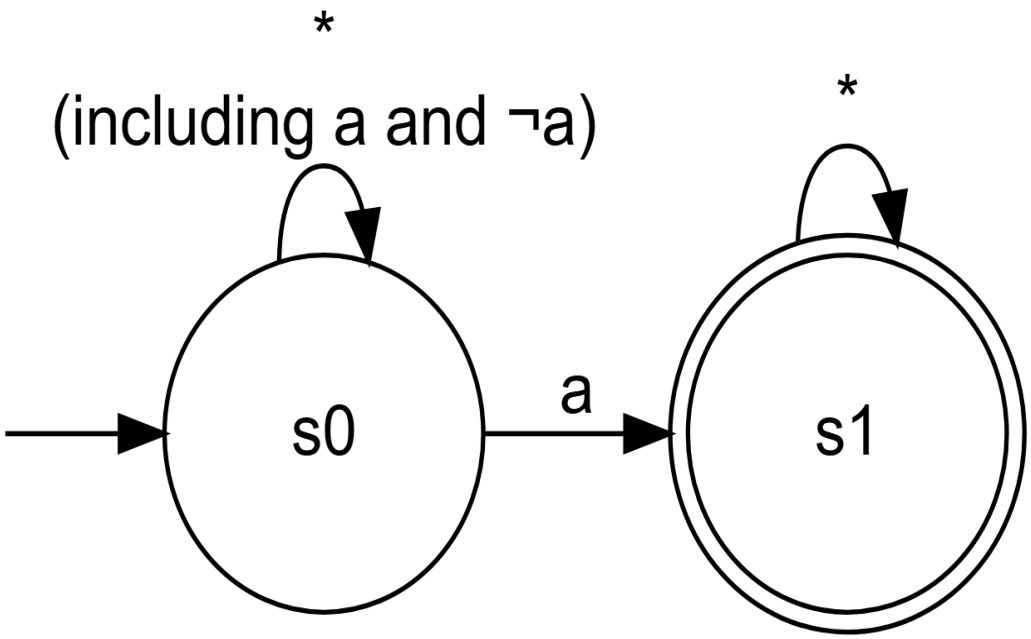
The formula $a \ \mathcal{U} \ b$ requires that a remain true until b becomes true (and b must eventually become true).



Operation: As long as $a \wedge \neg b$ is true, the automaton remains in the initial state s_0. As soon as b becomes true, it moves to the final state s_1 (and stays there) and the word is accepted. If $\neg a \wedge \neg b$ becomes true, it enters the sink state s_2 and the word is not accepted.

3. Eventually Operator: $\Diamond a$

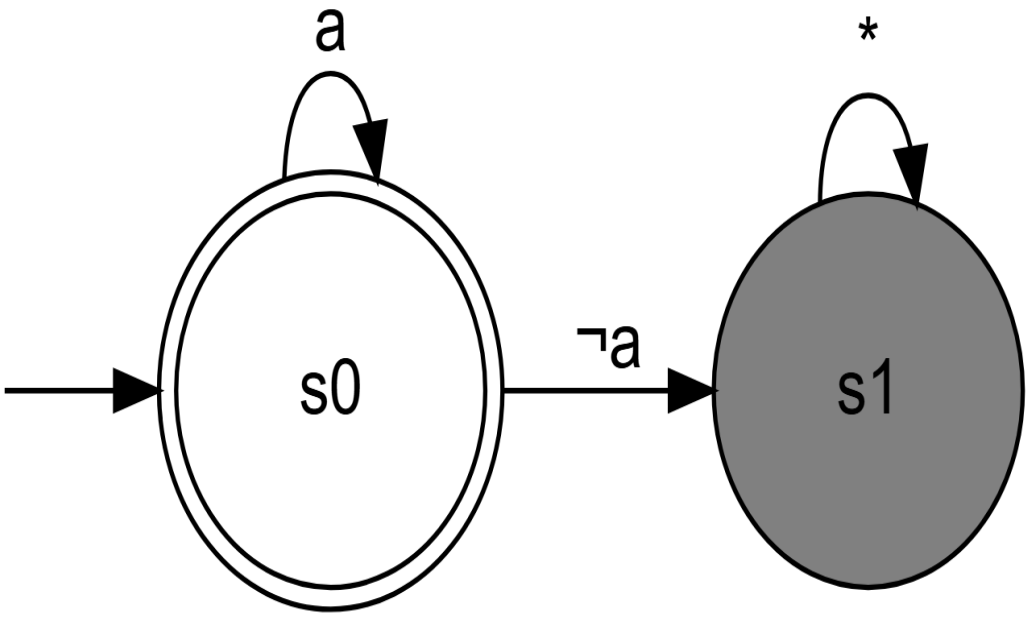
The formula $\Diamond a$ requires that a become true at some future moment.



Operation: The automaton remains in the initial state s_0. At some point, when a becomes true (not excluding that other a may have occurred), it moves to the final state s_1 and stays there, accepting the word. If a is never true, it remains indefinitely in s_0 and the word is not accepted (because s_0 is not final).

4. Always Operator: $\Box a$

The formula $\Box a$ requires that a be true at every instant.



Operation: The automaton starts in the final state s_0. As long as a is true, it remains in s_0. As soon as a becomes false, it enters the sink state s_1 and the word is not accepted. To be accepted, a must therefore be true at every instant.

These automata illustrate how the semantics of each operator translates into acceptance conditions on infinite executions. They form the basis for the systematic construction of automata for more complex LTL formulas.

Application to Model Checking

General Method

To verify that a system A (modeled by a Büchi automaton) satisfies a property F (expressed in LTL), we follow these steps:

1. Model the system with a Büchi automaton A.
2. Express the desired property as an LTL formula F.
3. Construct a Büchi automaton for the negation of F, denoted $\neg F$.
4. Compute the synchronous product of automata A and $\neg F$, which accepts the language $L(A) \cap L(\neg F)$.
5. Check if $L(A) \cap L(\neg F)$ is empty.

Interpretation of Results

- If $L(A) \cap L(\neg F) = \emptyset$, then $L(A) \subseteq L(F)$, which means that $A \models F$ (the system satisfies the property).
- Otherwise, any -word in $L(A) \cap L(\neg F)$ is a **counterexample**: an execution of system A that violates property F.

Emptiness Check

To check if $L(A) \cap L(\neg F)$ is empty, we use the algorithm based on strongly connected components (SCC):

- Construct the graph of the product automaton.
- Look for an SCC that is:
  1. Reachable from an initial state,
  2. Contains at least one final state.
- If such an SCC exists, then the language is non-empty (there is a counterexample). Otherwise, it is empty.

Simplified Example: An Elevator

Consider an elevator system modeled by an automaton A. A safety property could be: "if the elevator starts, then immediately after, it will remain moving until it stops". In LTL, this is expressed:

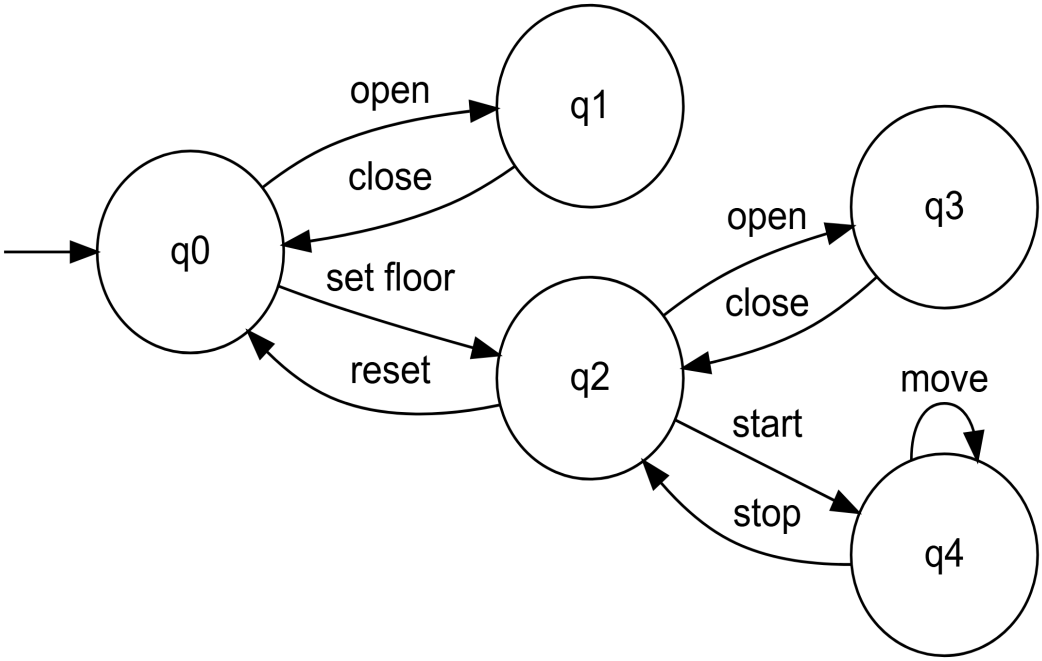
$$F = \Box\,(\text{start} \Rightarrow \bigcirc\,(\text{move}\ \mathcal{U}\ \text{stop}))$$

The negation is:

$$\neg F = \Diamond\,(\text{start} \wedge \bigcirc\,(\neg(\text{move}\ \mathcal{U}\ \text{stop})))$$

Step 1: System Modeling

Here is automaton A representing the simplified behavior of an elevator:



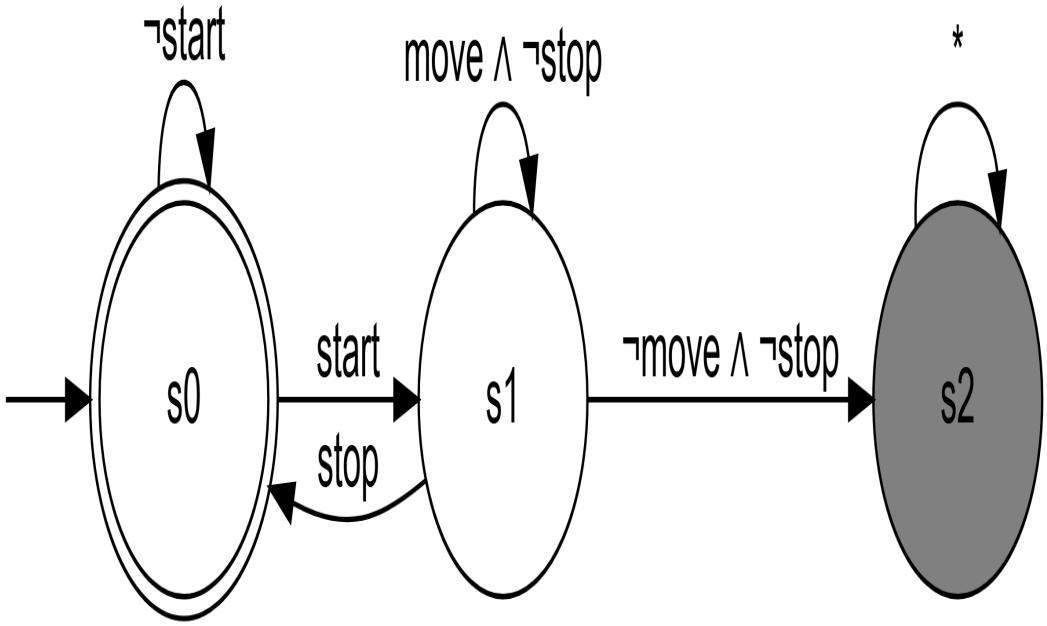
State Descriptions:

- q_0: Rest state (doors closed, no floor selected)
- q_1: Doors open
- q_2: Floor selected (waiting)
- q_3: Doors open after floor selection
- q_4: Elevator in motion

Step 2: Construction of Büchi Automata for F and $\neg F$

1. Büchi Automaton for $F = \square(\textbf{start} \Rightarrow \bigcirc(\textbf{move}\ \mathcal{U}\ \textbf{stop}))$

This automaton accepts executions where each occurrence of `start` is followed by a state where `move` remains true until `stop` becomes true.

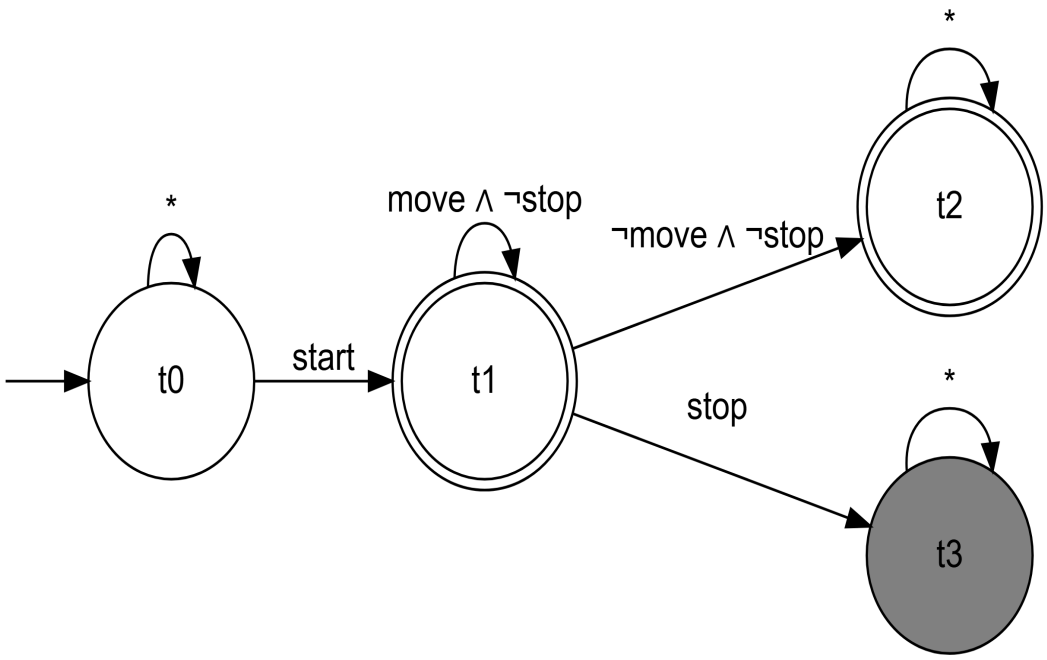


Operation:

- s_0 (final state): No current obligation
- s_1: A `start` just occurred, we must now verify move $\mathcal{U}$ stop
- **Acceptance condition**: Visit s_0 infinitely often
- An execution is accepted only if each obligation move $\mathcal{U}$ stop is eventually satisfied (by a `stop`)

2. Büchi Automaton for $\neg F = \Diamond(\textbf{start} \wedge \bigcirc(\neg(\textbf{move } \mathcal{U} \textbf{ stop})))$

This automaton accepts executions where at some moment, a `start` occurs and immediately after, move $\mathcal{U}$ stop is false.



Operation:

- t_0: We have not yet seen the problematic `start`
- t_1: We just saw `start`, we move to the next instant
- t_2 (final state): We verify $\neg$(move $\mathcal{U}$ stop); here as long as we have `move ¬stop`, we stay in t_2
- t_3 (final state): We have `¬move ¬stop`, which immediately violates move $\mathcal{U}$ stop
- **Acceptance condition**: Visit t_2 or t_3 infinitely often
- An execution is accepted if it remains indefinitely in t_2 (with `move ¬stop`) or in t_3

Step 3: Synchronous Product $A \times \neg F$

The product combines the states of A and $\neg F$. Let's analyze possible executions:

Problematic Execution:

1. $q_0 \xrightarrow{\text{set floor}} q_2$
2. $q_2 \xrightarrow{\text{start}} q_4$
3. $q_4 \xrightarrow{\text{move}} q_4 \xrightarrow{\text{move}} q_4 \xrightarrow{\text{move}} \ldots$

This execution corresponds to the word: `set floor, start, move, move, move, ...`

In the Product:

- After `start`, we go to t_2 in $\neg F$

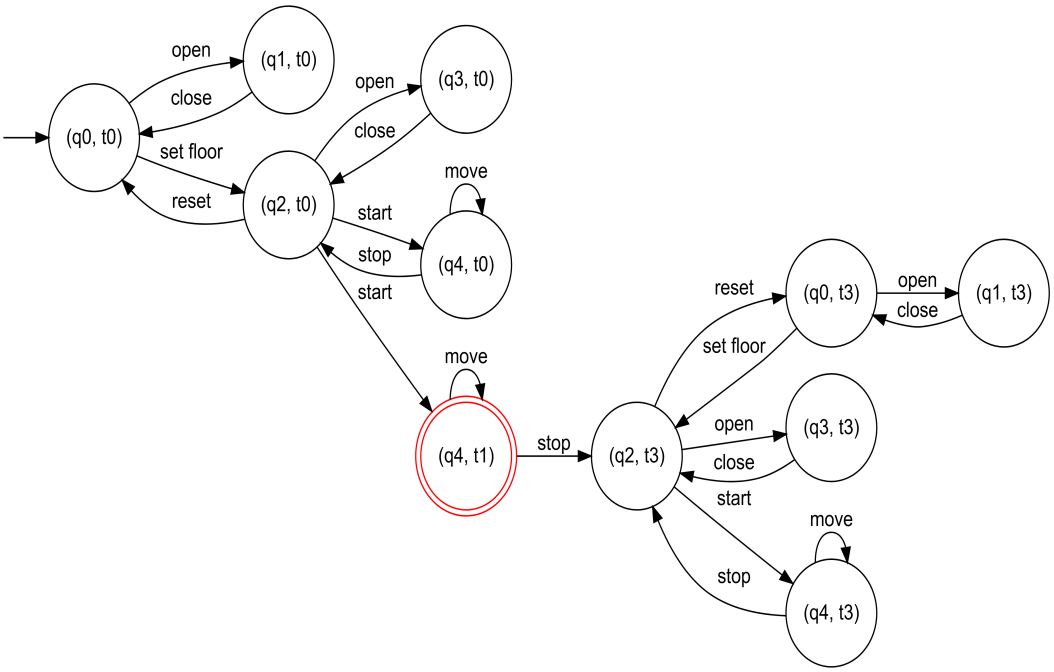
- In t_2, as long as we read `move ¬stop`, we stay in t_2
- Since the elevator can remain indefinitely in q_4 with `move`, we have an SCC (q_4, t_2) with the `move` loop

This SCC (q_4, t_2) is:

1. Reachable from the initial state
2. Contains a final state (t_2 is final)

Step 3: Synchronous Product $A \times \neg F$ (Detailed)

The synchronous product combines the states of A and $\neg F$. A state of the product is a pair (q_i, t_j) where q_i is a system state and t_j a state of the property-violating automaton.



Product Analysis:

1. **Final States**: Only (q_4, t_1) is final (represented by a red double circle), because t_1 is a final state in $\neg F$.
2. **Problematic Strongly Connected Component (SCC)**:
   - The state (q_4, t_1) forms an SCC with the transition $(q_4, t_1) \xrightarrow{move} (q_4, t_1)$
   - This SCC is reachable from the initial state via: $(q_0, t_0) \xrightarrow{\text{set floor}} (q_2, t_0) \xrightarrow{\text{start}} (q_4, t_1)$
   - This SCC contains a final state (t_1 is final)

Step 4: Emptiness Check

Since there exists an SCC (q_4, t_1) that:

1. Is reachable from an initial state
2. Contains at least one final state

Then $L(A) \cap L(\neg F) \neq \emptyset$.

Step 5: Counterexample

Since the language is not empty, we can extract a counterexample. The following execution is accepted by the product:

1. $(q_0, t_0) \xrightarrow{\text{set floor}} (q_2, t_0)$
2. $(q_2, t_0) \xrightarrow{\text{start}} (q_4, t_1)$
3. $(q_4, t_1) \xrightarrow{\text{move}} (q_4, t_1) \xrightarrow{\text{move}} (q_4, t_1) \xrightarrow{\text{move}} ...$

This execution corresponds to the infinite word: `set floor, start, move, move, move, ...`

Interpretation:

- The system A **does not satisfy** property F
- Reason: the elevator can start and continue moving indefinitely without ever executing the `stop` action
- To correct this, we would need to modify the system to guarantee that after a `start`, the elevator always eventually executes `stop` (for example by adding a timer or monitoring)

This example illustrates how LTL model checking can detect design flaws in systems.

Summary and Perspectives

Model checking is an automatic and complete method (under the assumptions of a finite system) for verifying temporal properties. The key points are:

- Modeling the system and properties with Büchi automata and LTL logic.
- Reducing the verification problem to an emptiness problem of intersection of -regular languages.
- Using graph algorithms (SCC) to decide emptiness.

However, complexity can be high (notably due to constructing the automaton for the negation of the LTL formula). Optimization techniques and industrial tools (like SPIN, NuSMV) are developed to apply model checking to real-size systems.